

Sistemas Operacionais

Threads:

- Definição
- Criação
- Sincronização



Windows



Mac OS



Linux



Chrome OS



Android



iOS

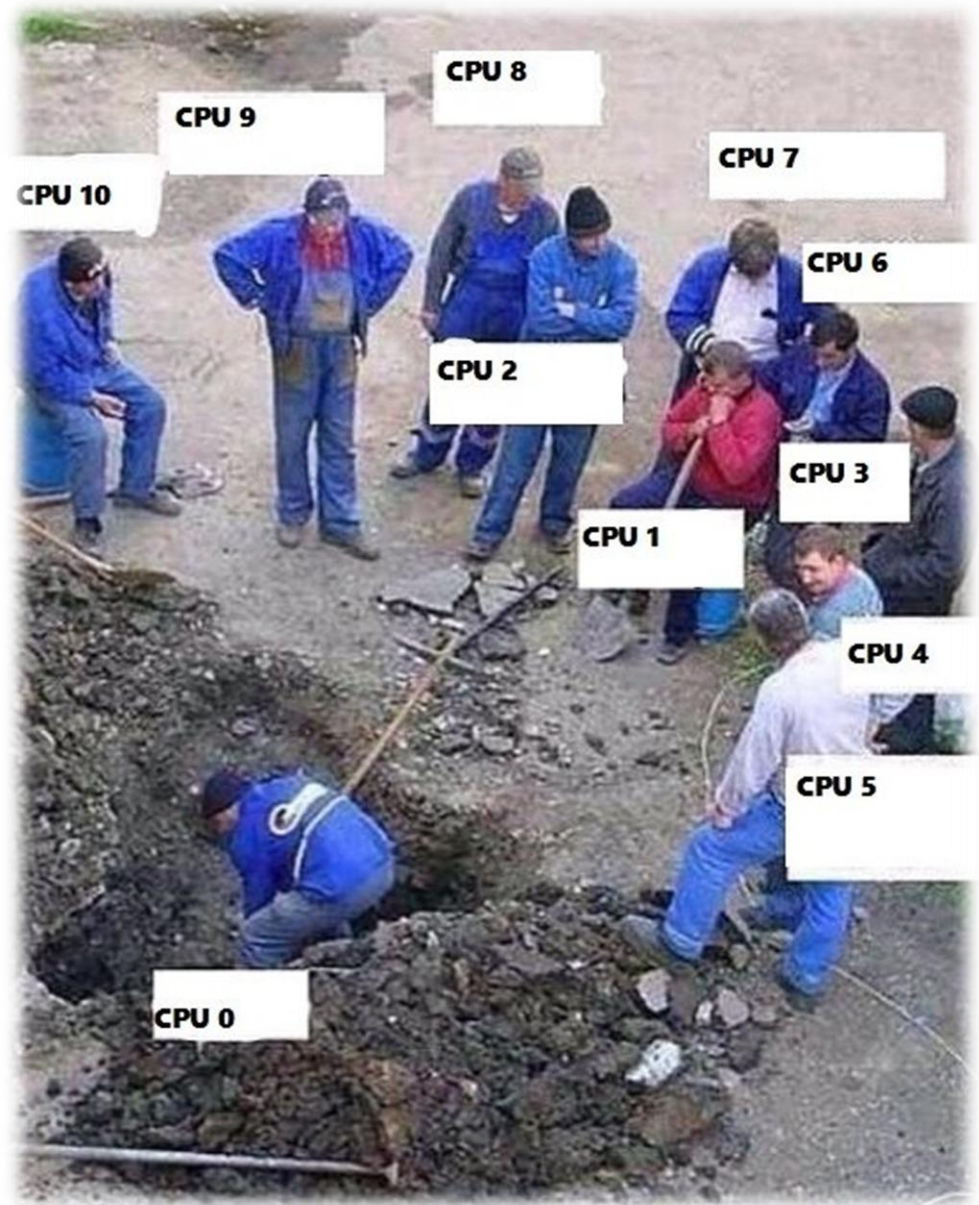


Unix



BSD

THREADS



Threads

O que são threads ?

Threads são múltiplos caminhos de execução que rodam concorrentemente na **memória compartilhada** e que compartilham os mesmos recursos e sinais do processo pai.

Uma thread é um processo simplificado, mais leve ou “light”, custa pouco para o sistema operacional, sendo fácil de criar, manter e gerenciar.

Casos em que o uso de threads é interessante:

- * Para salvar arquivos em disco.
- * Quando a interface gráfica é pesada.
- * Quando existem comunicações pela internet.
- * Quando existem cálculos pesados.

Threads

Conceito

- É um fluxo de execução **associado ao processo**.
- Tipo:
 - User thread
 - Kernel thread

Vantagens

- São mais fáceis de implementar.
- Sobreposição de tarefas na mesma aplicação.

Threads

- **USER THREAD:** Threads executando dentro de um processo são chamados de **threads de usuário** (*user-level threads ou simplesmente user threads*). Cada thread de usuário corresponde a uma tarefa a ser executada dentro de um processo.
- **KERNEL THREAD:** os fluxos de execução reconhecidos e gerenciados pelo núcleo do sistema operacional são chamados de **threads de núcleo** (*kernel-level threads ou kernel threads*).



Sobreposição de tarefas na mesma aplicação
(bom para app com muita entrada/saída)

THREAD POSIX

- PADRÃO IEEE 1003.1c

- Cada sistema operacional implementava sua própria versão de threads, Para superar esse problema foi desenvolvido o padrão POSIX 1003.1.c (Portable Operating System Interface) pelo IEEE (Institute of Electrical and Electronics Engineers)

- +60 FUNÇÕES

Chamada de thread	Descrição
pthread_create	Cria um novo thread
pthread_exit	Conclui a chamada de thread
pthread_join	Espera que um thread específico seja abandonado
pthread_yield	Libera a CPU para que outro thread seja executado
pthread_attr_init	Cria e inicializa uma estrutura de atributos do thread
pthread_attr_destroy	Remove uma estrutura de atributos do thread

Threads

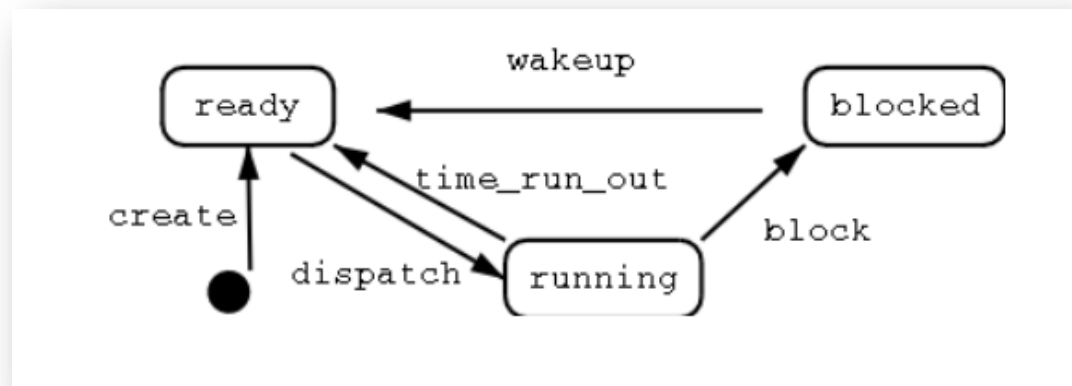
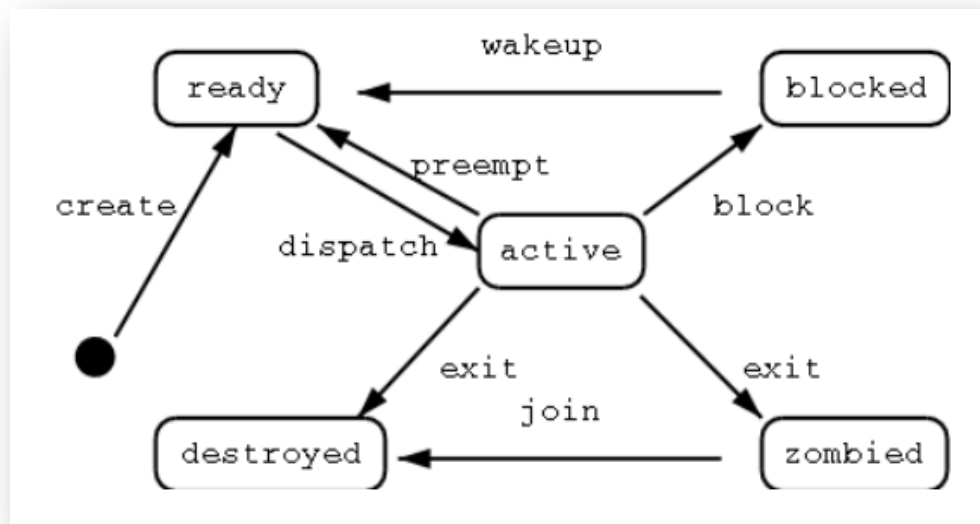
Existem diferentes categorias de sincronização, dentre as quais pode-se destacar:

Sincronização de dados: Threads concorrendo no acesso de uma variável compartilhada ou a arquivos.

Sincronização de hardware: Acesso compartilhado a tela, impressora.

Sincronização de tarefas: Condições lógicas

Processos X Threads



Criando Threads

```
int pthread_create(pthread_t *tid,  
                  const pthread_attr_t *attr,  
                  void * (* start)(void *),  
                  void *arg) ;
```

- 1) Cada novo thread é representado por um identificador (thread identifier ou tid) de tipo pthread_t.
- 2) O segundo parâmetro serve para indicar uma série de atributos e propriedades que o novo thread deverá ter.
- 3) O terceiro parâmetro indica a função de início do thread.
- 4) Finalmente o quarto parâmetro é o valor do argumento a ser passado à função de início, como seu parâmetro.
 - Uma thread retorna 0 no caso de sucesso e um código de erro no caso contrário.

Finalizando Threads

- Um thread termina quando a função de início, indicada quando da criação, retornar, ou quando o próprio thread invocar o serviço de terminação:

```
pthread_exit(void *value_ptr) ;
```

Onde **value_ptr** é o ponteiro que o thread deve ter como resultado.

Sincronização de Threads

```
int  pthread_join (pthread_t thread,  
                  void ** status);
```

- O comando **join é executado para esperar o fim de uma Thread**, de maneira semelhante a `wait()` para processos (bloqueia o thread que *chamou* esta função até que o thread com o identificador *thread_id* termine.).
 - O Identificador da thread deve ser especificado.

Sincronização de Threads

- **Deadlock** ocorre quando acontece um **impasse**, e dois ou mais processos ficam impedidos de continuar suas execuções, ou seja, ficam bloqueados, esperando uns pelos outros.

